МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Кафедра прикладної математики

ЗВІТ ЛАБОРАТОРНОЇ РОБОТИ №3

з дисципліни «Бази даних»

на тему: «Міграція Бази Даних»

Виконав:
студент III курсу групи КМ-32,
спеціальність F1
Прикладна математика
Мурсюкаєв Дамір Андрійович

Київ – 2026

Технології:

Мова: Python 3

БД: PostgreSQL (основна) → MySQL (цільова)

ORM: SQLAlchemy 2.0

Міграції: Alembic 1.13

1. ORM модель

Описано дві моделі: Weather та AirQuality.

Модель Weather містить всі обов'язкові типи колонок:

- country – String (текст);
- wind_degree – Integer (ціле);
- wind_kph – Float (дробне);
- wind_dir – Enum WindDirection (16 напрямків: N, NNE, NE тощо);
- last_updated – Date (дата);
- sunrise – Time (час).

Обрана категорія (10 % 4 = 2) – стан повітря (air quality). Всі її колонки описано в моделі AirQuality: air_co, air_no2, air_ozone, air_so2, air_pm25, air_pm10, air_epa_index, air_defra_index.

2. Міграція

Реалізовано 3 послідовні міграції:

Міграція 001 – створення початкової таблиці weather з усіма колонками включно з повітрям.

Міграція 002 – рефакторинг: виокремлення колонок стану повітря в окрему таблицю air_quality з Foreign Key на weather.id. Дані переносяться через INSERT INTO … SELECT FROM, після чого зайві колонки видаляються з weather.

Міграція 003 – додавання булевої колонки go_outside до air_quality.

Результат alembic history:

002 -> 003 (head)

001 -> 002

<base> -> 001

Поточний стан: 003 (head)

Структура таблиць у PostgreSQL після всіх міграцій:

weather: id, country, wind_degree, wind_kph, wind_dir, last_updated, sunrise;

air_quality: id, weather_id (FK), air_co, air_no2, air_ozone, air_so2, air_pm25, air_pm10, air_epa_index, air_defra_index, go_outside.

3. Колонка “Чи варто виходити на вулицю”

Формула базується на трьох показниках якості повітря:

- EPA Index ≤ 3 (рівень “добре” або “помірно”);
- PM2.5 < 35 мкг/м³;
- Ozone < 180 мкг/м³.

Якщо всі три умови виконано – go_outside = true, інакше false.

Реалізовано у двох місцях:

1. В міграції 003 – SQL UPDATE для заповнення існуючих даних;
2. В WeatherService.calculate_go_outside() – для нових записів під час завантаження CSV.

Статистика по базі:

total: 136 853

safe: 111 761(go_outside = true)

unsafe: 25 092 (go_outside = false)

Приклад даних з air_quality:

id=5, pm25=183.4, epa=5, go_outside = false (забруднено)

id=1, pm25=8.4, epa=1, go_outside = true (чисте)

4. Користувацький інтерфейс

Три шари:

- Repository (repository.py) – запити до БД через SQLAlchemy;
- Service (service.py) – бізнес-логіка, валідація дати, формула go_outside;

- CLI (cli.py) – взаємодія з користувачем.

Користувач вводить: країну, дату, опціонально максимальний EPA Index. Виводиться повна погодна інформація та вердикт "Чи виходити?".

5. Міграція з PostgreSQL на MySQL

Реалізовано в migrate_db.py. Схема створюється через Base.metadata.create_all(), дані копіюються батчами по 500 записів через ORM-сесії обох БД паралельно.

Перевірка – однакова кількість записів в обох СУБД:

PostgreSQL: weather=138853, air_quality=136853

MySQL: weather=138853, air_quality=136853

go_outside у MySQL зберігається як 0/1 (булеан в MySQL), що є коректним.

Питання до роботи:

1) Чи можна завантажувати всю базу, а описувати в ORM моделі тільки деякі колонки?
Так. SQLAlchemy маппить лише ті колонки, які оголошені в моделі. Решта ігнорується при читанні та не вставляється при записі.

2) Чи є дефолтне значення при створенні у колонці “Чи варто виходити на вулицю”? Поясніть?
Ні. Колонка створюється з nullable=True, тобто дефолт – NULL. Це правильно, бо значення розраховується формулою – ставити false або true за замовчуванням було б семантично некоректно.

3) У скільки етапів ви зробили 3тє завдання?
У два етапи: спочатку міграція 003 додає колонку й заповнює існуючі рядки через SQL UPDATE, потім під час завантаження CSV нові рядки отримують значення через WeatherService.calculate_go_outside().

4) На яку частину застосунку ви поклали фунцію заповнення колонки “Чи варто виходити на вулицю” даними?
На Service layer – WeatherService.calculate_go_outside(). Це відповідає архітектурі: бізнес-логіка не повинна бути ні в репозиторії, ні в CLI.

5) Чи легко було накочувати міграції, перейти з однієї бази на іншу? Які проблеми траплялись?
Загалом Alembic зручний. Основні труднощі: синтаксис UPDATE … FROM у PostgreSQL не сумісний з MySQL (у downgrade міграції 002); для Enum потрібно native_enum=False щоб він зберігався як VARCHAR і працював в обох СУБД – це враховано в моделі.

Репозиторій github: https://github.com/Damikers/DB-Lab3